

Proyecto de Investigación — Programación Lógica y Funcional 2026 “B”

Bloque 1: Programación Funcional (nivel introductorio) — 40 temas de investigación y
rúbrica de 5 categorías

TecNM Campus Tijuana — Ingeniería en Sistemas Computacionales (ISC-2006)

Agosto 2026

Presentación

Esta es la **primera lista de temas de investigación** del curso Programación Lógica y Funcional 2026 “B”. Cubre exclusivamente el **paradigma funcional** (Erlang/OTP, Elixir, Haskell, OCaml, Clojure, Scala, Gleam). Los temas de **programación lógica** (Prolog, Datalog, ASP, CLP(FD)) se publicarán en una lista posterior.

Los 40 temas son de **nivel introductorio**: historia y antecedentes del paradigma, conceptos fundamentales y primer contacto con los lenguajes. Están pensados para las primeras semanas del curso, antes de abordar tipos avanzados, mónadas o concurrencia OTP.

Cada estudiante desarrollará **una investigación técnica individual**, la publicará mediante el flujo **Fork — Pull Request** y documentará el uso de asistentes de IA (LLM).

Los 40 temas de esta lista fueron **revisados para no duplicarse** con las investigaciones ya integradas en **research/**:

- *Prolog y modelos de lenguaje grandes: aproximación neuro-simbólica* (paradigma lógico),
- *Erlang/OTP y LLMs: arquitectura tolerante a fallos* (queda excluido como tema).

No se admiten temas repetidos ni variantes menores de un tema ya trabajado; el docente asigna o confirma el tema por Google Classroom.

Entrega esperada

Carpeta personal dentro de **research/<nombre-del-tema>/** con:

- **README.md** — título, introducción, desarrollo técnico (mínimo 500 palabras), conclusiones y bibliografía en formato **IEEE**.
- **anexo.md** — bitácora de uso de LLM: prompts reales, resultados obtenidos y reflexión crítica (¿ayudó?, ¿hubo sesgos o errores?). Obligatorio si se usó IA.
- (*Opcional*) código que compile/ejecute, diagramas y PDF de papers de referencia.

Reglas del flujo Fork — Pull Request

- No cambiar la ruta indicada ni renombrar **README.md**.
- No modificar archivos ajenos ni el **README.md** de otras carpetas.
- Un Pull Request por estudiante, con commits claros y descriptivos.
- El PR que altere la estructura del repositorio se **rechaza** sin calificación.

Ética en el uso de IA

El uso de asistentes de IA se rige por la guía [AI_GUIDANCE.md](#) del repositorio. Es **obligatorio** leerla antes de entregar. En resumen: la IA es apoyo, no autora; todo lo generado se revisa, se entiende y se cita; y su uso se declara en [anexo.md](#).

Reglas de contenido (según [CLAUDE.md](#) y [casos_reales_mundo_real.md](#))

- Todo ejemplo de código debe compilar/ejecutar; el pseudocódigo se etiqueta como tal.
- Erlang: usar comportamientos OTP (`gen_server`, `supervisor`), nunca `spawn` sin supervisión.
- Haskell: `Maybe`/`Either` para errores, nunca funciones parciales sobre listas arbitrarias.
- Casos de industria: solo afirmaciones verificables con fuente (WhatsApp/Erlang, Discord/Elixir, Nubank/Clojure, Jane Street/OCaml, Standard Chartered/Haskell). No atribuir a organizaciones locales (IMSS, SAT, maquiladoras) un stack concreto sin fuente directa.

Los 40 temas — Bloque Funcional introductorio (2026 “B”)

Historia y antecedentes

1. El cálculo lambda de Alonzo Church (1936) como fundamento teórico de la programación funcional
2. Lisp (John McCarthy, 1958): el primer lenguaje funcional y sus ideas duraderas
3. De ISWIM a ML: la línea que llevó a los lenguajes funcionales tipados
4. Historia de Haskell: por qué un comité creó un lenguaje “puro y perezoso” en 1990
5. Historia de Erlang: cómo Ericsson resolvió la tolerancia a fallos en telefonía (1986)
6. Miranda, Hope y los lenguajes funcionales de los años ochenta
7. Línea de tiempo de la programación funcional: de 1930 a Gleam (2024)
8. John Backus y su conferencia Turing de 1977: “¿Puede liberarse la programación del estilo von Neumann?”

Conceptos fundamentales

9. ¿Qué es un paradigma de programación? Imperativo, orientado a objetos, funcional y lógico
10. Programación declarativa frente a imperativa: describir “qué” en lugar de “cómo”
11. Funciones puras: definición, ejemplos y contraejemplos
12. Transparencia referencial y por qué facilita razonar sobre el código
13. Efectos secundarios: qué son y por qué la programación funcional busca controlarlos
14. Inmutabilidad: datos que no cambian y qué implica para el programador
15. Expresiones frente a sentencias: programar evaluando en lugar de ordenar
16. Ligado de nombres frente a asignación destructiva de variables
17. El concepto de estado y cómo lo maneja un lenguaje funcional

Funciones como valores

18. Funciones de primera clase: pasar y devolver funciones como cualquier dato
19. Funciones de orden superior: **map**, **filter** y **reduce** explicadas desde cero
20. Funciones anónimas y expresiones lambda
21. Composición de funciones: construir programas encadenando funciones pequeñas
22. Currificación y aplicación parcial: una introducción con ejemplos
23. *Closures* (clausuras): funciones que recuerdan su entorno
24. El operador *pipe* de Elixir (`|>`) y la lectura de izquierda a derecha

Recursión

25. Recursión como alternativa a los bucles: caso base y caso recursivo
26. Factorial, Fibonacci y sumatoria: los ejemplos clásicos paso a paso
27. Recorrer listas de forma recursiva
28. Introducción a la recursión de cola y por qué importa
29. Errores comunes al aprender recursión: falta de caso base y desbordamiento de pila

Datos y coincidencia de patrones

30. Listas enlazadas inmutables: **cons**, cabeza y cola
31. Tuplas y registros: agrupar datos sin clases
32. *Pattern matching* (coincidencia de patrones): una introducción
33. Tipos algebraicos de datos sencillos: enumeraciones y variantes
34. Representar la ausencia de valor sin **null**: **Maybe**, **Option** y **nil**

Primer contacto con los lenguajes

- 35. Primeros pasos en Haskell con GHCi
- 36. Primeros pasos en Elixir con IEx y Mix
- 37. Primeros pasos en Clojure y la programación dirigida por REPL
- 38. Instalación y configuración de entornos funcionales (Erlang, GHC, Elixir, Clojure)

Contexto e industria

- 39. Programación funcional en lenguajes de uso común: JavaScript, Python y Java Streams
- 40. Dónde se usa la programación funcional hoy: WhatsApp (Erlang), Discord (Elixir) y Nubank (Clojure) — panorama introductorio

Rúbrica de evaluación — 5 categorías (100 puntos)

#	Categoría	Pts	Qué se evalúa
1	Rigor técnico y profundidad	30	Comprensión correcta y profunda del tema; exactitud de los conceptos; fuentes actualizadas y pertinentes; desarrollo técnico de al menos 500 palabras con datos, comparativas o ejemplos que compilan/ejecutan.
2	Estructura y claridad del README.md	20	Uso correcto de Markdown; organización lógica (introducción, desarrollo, conclusiones); redacción profesional y ortografía sin faltas graves.
3	Originalidad y análisis crítico	20	Síntesis y redacción propias; el texto no es una copia de la salida de un LLM; hay interpretación, comparación entre lenguajes o paradigmas y postura argumentada del estudiante.
4	Uso del repositorio y flujo Fork — Pull Request	15	Ruta y nombre de carpeta correctos; README.md sin renombrar; no se tocan archivos ajenos; commits claros; un solo PR bien descrito.
5	Bitácora de IA (anexo.md) y bibliografía IEEE	15	anexo.md con prompts reales, resultados y reflexión honesta sobre el uso de IA; bibliografía con fuentes confiables (IEEE, libros, papers, sitios oficiales) y formato IEEE correcto.

Escala de desempeño por categoría

Nivel	Porcentaje de la categoría	Descripción
Excelente	90–100 %	Cumple todos los criterios con evidencia sólida y sin observaciones.
Satisfactorio	75–89 %	Cumple lo esencial con observaciones menores.
Suficiente	60–74 %	Cumple parcialmente; faltan elementos o hay imprecisiones.
Insuficiente	0–59 %	No cumple el criterio mínimo o hay copia sin análisis.

Penalizaciones

- **Tema duplicado** o variante menor de una investigación ya integrada en **research/**: se devuelve el PR sin calificar hasta reasignar tema.

- **Afirmación de industria sin fuente verificable** (atribuir un stack FP a una organización sin cita directa): categoría 1 con penalización según gravedad.
- **Código que no compila/ejecuta** presentado como funcional: penalización en categoría 1.
- **Alteración de la estructura del repositorio** o de archivos ajenos: PR rechazado (categoría 4 en 0).
- **Uso de IA no declarado** detectado: categorías 3 y 5 en 0 y reporte de integridad académica según [AI_GUIDANCE.md](#).
- **Entrega tardía**: según las políticas del curso.

Documento del curso Programación Lógica y Funcional (ISC-2006), semestre 2026 “B”. Bloque 1 de 2: Programación Funcional. El bloque de Programación Lógica se publicará por separado. Referencias del curso: [SYLLABUS.md](#), [README.md](#), [AI_GUIDANCE.md](#), [casos_reales_mundo_real.md](#).